CODE APEX

THE END IS COMING.

POWERED BY verzii

Team Details

- Team name - **Mac_Enthu**
- Team leader Name - **Aayush Aade**
- Problem Statement (Domain) - **Track 1 (Emerson) · PS 1.2 Project IO SIGNAL SUDOKU**

SlotSmith · rule-driven, independently checked, revision-aware DeltaV I/O allocation

Places each signal in cabinet → CIOC → baseplate → CHARM under project rules and re-checks the export independently. Prototype, synthetic data.

Problem Statement & Target Need

Who is affected

DeltaV project and I/O engineers must give every field signal an exact home: cabinet → CIOC → baseplate → CHARM slot. Panel builders and site teams then wire to that allocation.

The current gap

Emerson's PS: 10,000+ signals are allocated by hand in Excel, and each customer revision means reworking it and tracing the impact again. DeltaV tools automate the steps around it (auto-sense, late binding, Bulk Edit); we found no public tool that places signals from project rules (internal tools may exist).

Why it matters now

Electronic Marshalling makes late changes physically cheap ("without lifting a wire"), **but allocation planning and revision tracking remain a manual workflow burden (per the PS).** Six interacting rules must hold at once, on every revision, with a technical query (TQ) for each data conflict.

Sources: Emerson PS #02 p.4; DeltaV Electronic Marshalling PDS (Aug 2026); Smart Commissioning WP (2021) p.5; Bulk Edit WP (2022).

THE PUZZLE, PER EMERSON'S RULEBOOK

Cabinet ≤ 4 CIOCs → **CIOC** 8 baseplates → **Baseplate** 12 CHARMs → **CHARM** 1 signal

ONE INTERACTION two 6-pair cables, one on side A, one on side B

Would fit one baseplate… → …so each gets its own CIOC



× A and B never share a CIOC

Rule 4 keeps each cable whole, in pair order, so both baseplates stay half empty. Capacity (rule 3) and the spare reserve (rule 5) must absorb that.

SIX RULES, ELEVEN INDEPENDENT CHECKS

1 Groups together **G-1**	2 A and B apart **S-1**	3 Capacity 12·8·4 **C-1**
4 Cable intact **K-1**	5 ~10% spare **P-1**	6 CHARM + block **H-1..6**

+5 checks on data and output: Q-1 every signal placed or queried · D-2 one signal per slot · N-1..6 names · C-2 controllers · M-1 rows match the MDB

Proposed Solution & Innovation

What makes it different

- **Independently checked:** a separate module re-reads the exported file (11 checks).
- **Revision-aware:** unchanged cables stay put; CP-SAT repairs a local window.
- **Configurable:** rule parameters live in JSON; new rule types may need code.
- **Traceable:** every TQ cites a rule ID and the file, sheet and row.

Core mechanism

Baseline allocation is constructive, with no solver: each cable becomes one block of 24/12/6/2 slots and first-fit-decreasing (FFD) packs blocks per group and A/B side into baseplates, CIOCs and cabinets. CP-SAT enters only to repair a revision (10 s limit per window) and reports each window as proven exact or best found.

Why it's feasible in this timeframe

Open-source Python, OR-Tools and openpyxl; 122 automated tests pass. The prototype allocates and verifies 20,033 synthetic signals in about 6 s on a laptop (slide 6). AI helpers are planned and would only suggest; none runs today.

WORKFLOW

1 Input	Excel or CSV lists; columns auto-mapped; file · sheet · row kept
2 Validate	Missing, duplicate or conflicting data → TQs with rule ID
3 Allocate	Constructive first-fit-decreasing packing, no solver
4 Verify	Separate module re-reads the exported file; 11 checks
5 Revise	Pin unchanged, CP-SAT window repair, impact sheet

PROVED	First-fit-decreasing uses the fewest baseplates for one A/B side with cable sizes 2/6/12.
TESTED	Group baseplate totals matched exact CP-SAT optima in all 19,682 small cases and 148 of 150 larger groups (2 unproven, none worse).
NOT CLAIMED	Minimum CIOCs, cabinets or revision moves; global optimality; results on organiser data.

System Architecture & Tech Stack

TECH STACK

Frontend	Streamlit workbench (built): 7 stages, cabinet map, TQs, revision diff
Backend	Python 3.12 · pandas · openpyxl · pydantic typed rule sets · CLI
Database	No database: xlsx / csv in, JSON rule sets, xlsx workbooks out
AI / APIs	OR-Tools CP-SAT for revision repair only. No LLM runs today; an optional helper is planned
Hosting / DevOps	Local run · pytest + Hypothesis · Docker bundle planned

DATA FLOW

solid = built · amber = verification · dashed = planned

Rule set JSON, typed with pydantic · read by validate, allocate, revise and verify

Rev A list — xlsx or csv, one or many files

→ **Ingest** — map columns, keep file·sheet·row

→ **Validate** — rules + data → TQs

→ **Allocate** — FFD packing → baseplates, CIOCs, cabinets

→ **Export** — writes the workbook sheets

→ **Workbook** — sheets: allocation, summary, TQs, rule check

→ **Verify** — 11 checks on the exported file

Revisions add a Revision Impact sheet: added, deleted, moved, modified.

Rev B list — xlsx or csv

→ **Revision engine** — pin unchanged → greedy fill → CP-SAT window repair → full re-run only as fallback

→ Export

SURFACES, TESTS AND STORAGE

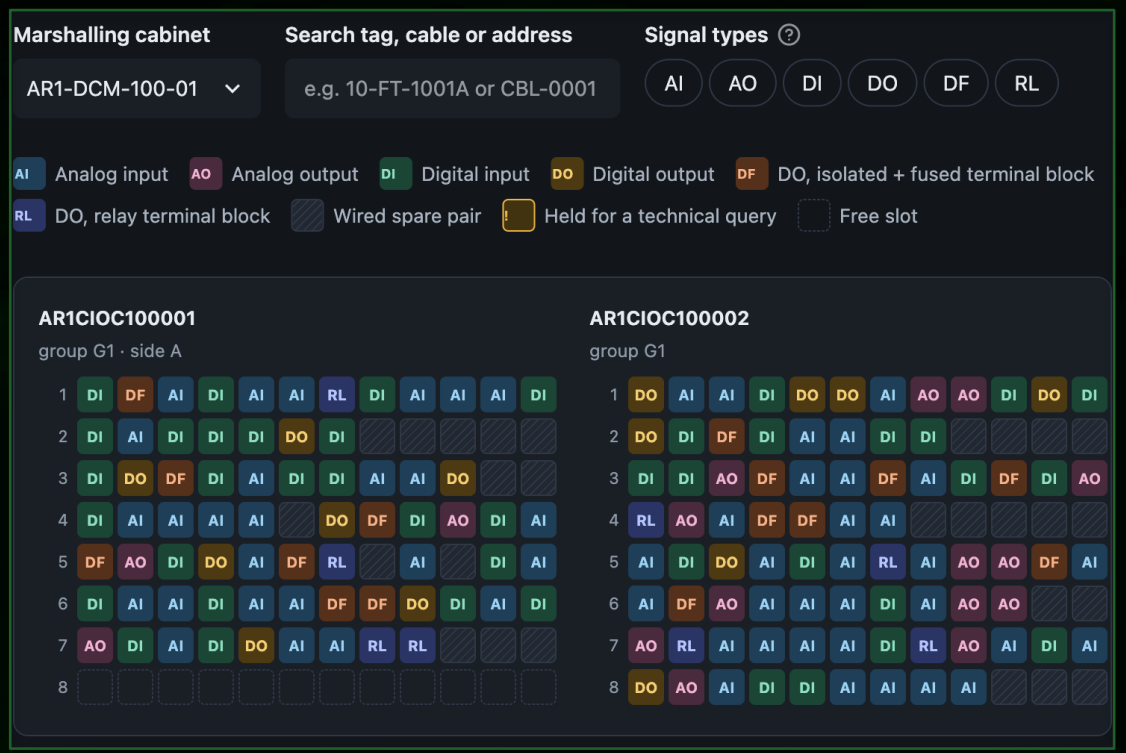
- **Streamlit** — 7-stage workbench
- **CLI** — slotsmith run / revise
- **pytest** — 122 tests
- **Files only** — no database
- **LLM helper** — planned
- **Docker** — planned

The verifier imports only the rule set and parses exported names back into positions.

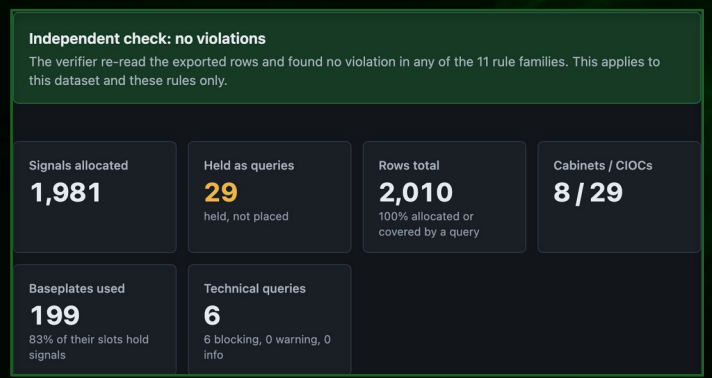
Key Features & UI/UX

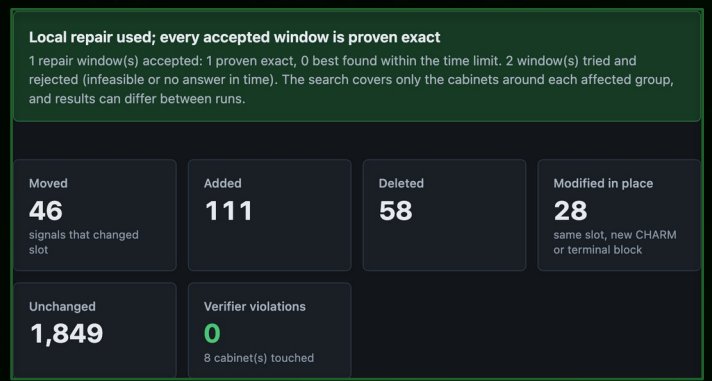
WORKBENCH · 7 STAGES BUILT

1 · Input | 2 · Review | 3 · Allocate | 4 · Results | 5 · Cabinets | 6 · Revise (Rev B) | 7 · Deliverables



Cabinet inspector. Every slot shows its signal type; hatched = wired spare, ! = held for a TQ. 2 of 4 CIOCs of cabinet AR1-DCM-100-01.





RESULTS

0 violations

Verifier re-read the exported rows: all 11 rule families pass, for this dataset and these rules only.

29 rows held under 6 blocking TQs; every row placed or queried.

REVISE (REV B)

46 moved

of 1,981 allocated signals; 111 added, 58 deleted, 28 modified in place, 0 violations.

Repair window proven exact in this run; runs can differ.

Real screenshots of the current build, captured 11 Oct 2026. Data is SYNTHETIC: 2,000-signal MDB with 6 injected errors plus a synthetic Rev B. Not customer data.

Implementation Roadmap & Scope

What's built today, what's in progress, and what's planned beyond the hackathon

BUILT · AS OF 11 OCT — Built

- Engine: ingest to verified export
- Revision engine + CP-SAT repair
- Streamlit workbench, 7 stages
- 122 automated tests pass

ROUND 2 · 15–18 OCT — Planned

- Map export to organiser template, if released
- Confirm open rule readings with mentors
- Reduce CP-SAT run-to-run variance
- UI: mapping review and override

GRAND FINALE · 24 OCT — Planned

- Run on organiser samples + hidden test
- AI mapping suggestions, user-reviewed
- Demo hardening, offline bundle

BEYOND — Idea, untested

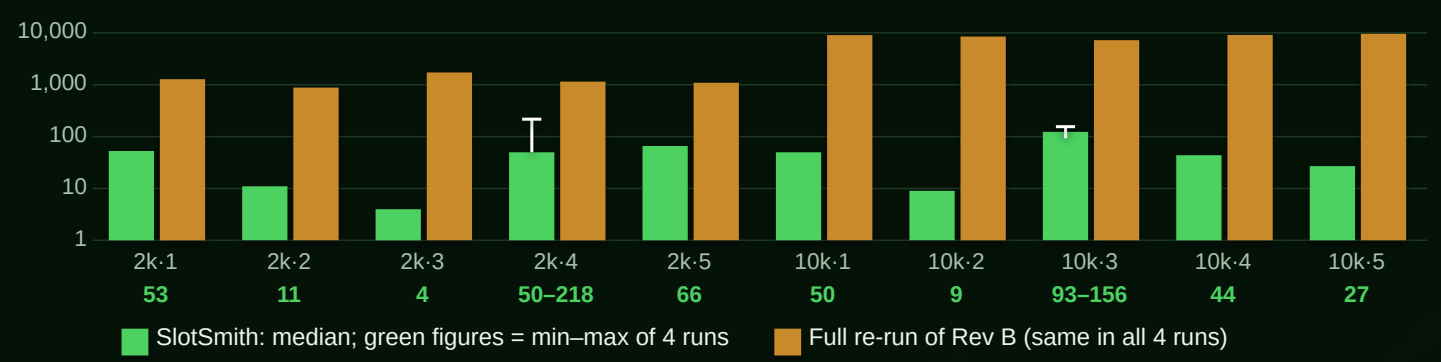
- Bulk-Edit-style export (format unvalidated)
- Other electronic-marshalling platforms
- Cabinet labels and drawings

EVIDENCE · SYNTHETIC DATA ONLY

Apple M5 laptop, 16 GB · not evidence about the hidden test

Revision cost: signals moved per case, log scale

total 429–575 over 4 runs vs 49,805 for a full re-run



0
verifier violations
12 allocation runs, 10 revisions, 500 to 20,033 signals

81/81
injected errors raised as TQs
3 synthetic MDBs, 27 errors each

0
baseplates over the exact minimum
closed form, sizes 2/6/12, 12 runs

7 of 11
repair windows proven exact
4 best-found; 7 of 10 revisions used CP-SAT

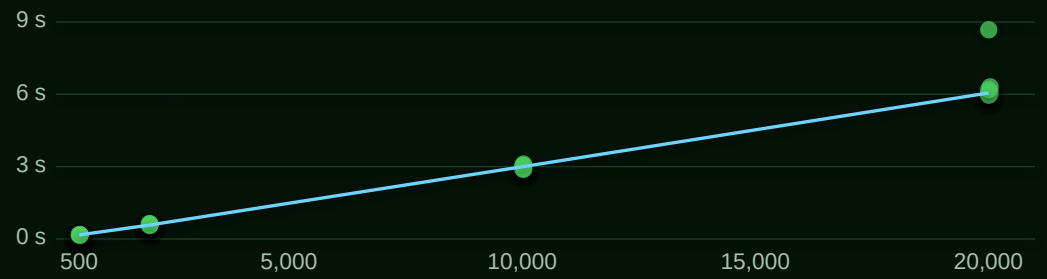
Event dates from the Code Apex 3.0 handbook (subject to change); roadmap items are intentions, not commitments. The re-run baseline re-allocates Rev B from scratch with the same engine; it is a naive reference, not an optimum.

Expected Impact & Scalability

Who benefits, how big the opportunity is, and how this grows beyond a demo.

MEASURED TODAY · SYNTHETIC

end-to-end time vs signals



Dots: every allocation run in 3 batches (3 per size); line: medians. 20,033 signals took about 6 s on one laptop (batch medians 6.0, 6.0, 6.3 s; one 8.7 s outlier). Observed, not a complexity proof. Allocation alone is about 0.01 s; the rest is Excel I/O, validation and verification.

EXPECTED BENEFITS · NOT MEASURED

IO / project engineers

Allocation becomes a re-runnable, rule-checked step instead of hand-editing 10,000+ rows.

Reviewers and QA

Every export carries a rule-check report; every placement and TQ traces to a file, sheet and row.

Customers and EPCs

Specific, traceable TQs and a Rev A → Rev B impact list.

No time or cost saving has been measured, so none is claimed.

SCALE CONTEXT · SOURCED

10,000+

signals per project, allocated by hand today (Emerson PS)

≤ 28,800

CHARM channels per DeltaV system (300 CIOCs × 96): a spec maximum, not a typical project

WHERE IT FITS

DeltaV tools
auto-sense, late binding, Bulk Edit

→

SlotSmith
allocation planning, checking, Rev B impact

Complements DeltaV tools; it does not replace them. Export adapters to their import formats and other marshalling platforms are untested ideas.

LIMITS · NEXT VALIDATION

- **Next dependency: the organiser's sample MDB and allocation template (not released).**
- Synthetic data only; no official-template compatibility claimed.
- Rule readings (straddling, spare rounding, controller mapping) await mentors.
- New rule types need code; CP-SAT repair is window-local and varies between runs.

CODE APEX

BUILD BEFORE IT DOES.

POWERED BY verzii

THANK YOU

Allocation you can re-run, check independently, and revise without starting over.

Team **Mac_Enthu** · Aayush Aade (lead) · Siddhant Patil · Gaurang Bagwe

LINKS

GitHub	https://github.com/Sidd927/SlotSmith
Demo video	**{{YOUTUBE_DEMO_URL}}**
Documentation	**{{DOCUMENTATION_URL}}**